

Experiment: 01

1. Objectives:

- To understand the concept of the Newton Raphson Method using python.
- To learn and observe the convergence process.

2. Introduction:

The purpose of Newton's method is to find a root of a function. The idea is to start with an initial guess at a root, approximate the function by its tangent line near the guess, and then take the root of the linear approximation as a next guess at the function's root. This will typically be closer to the function's root than the previous guess, and the method can be iterated.

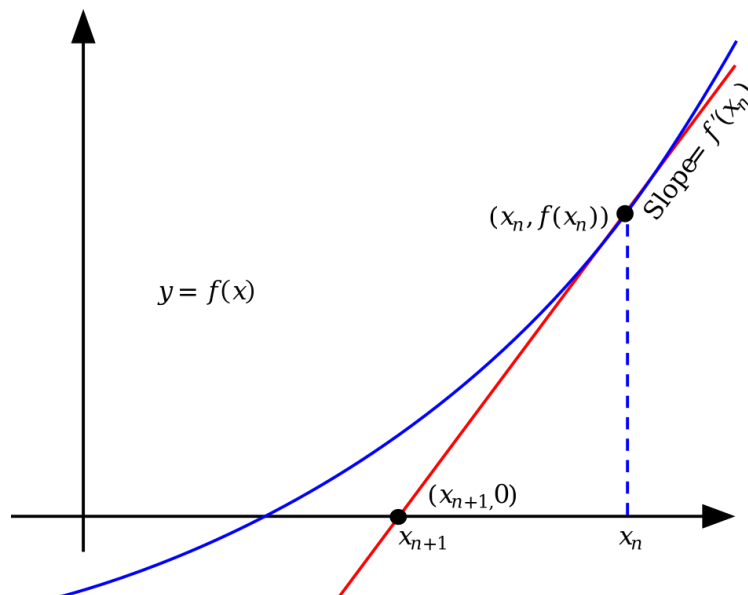


Fig 1: Newton Raphson Method

3. Procedure

Algorithm: Newton Raphson Method in Python

1. Start
2. Define $f(x)$ and $f'(x)$.
3. Take initial guess x_0 , tolerance, and max iterations.
4. Repeat until convergence or max iteration:
5. $x_1 = x_0 - \{ f(x_0) / f'(x_0) \}$
6. If $|x_1 - x_0| \leq \text{tolerance}$ then stop.
7. Else set $x_0 = x_1$.
8. Print root.
9. End

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** PyCharm Community Edition 20251.1.1

5. Experiment Implementation:

Code:

```
import math

def f(x):
    return 3 * x - math.cos(x) - 1

def df(x):
    return 3 + math.sin(x)

x0 = 1
tol = 0.0005

def newrap(x0,tol,max_i=100):

    i=1

    while i<max_i:
        fx=f(x0)
        dfx=df(x0)
        x1=x0-(fx/dfx)
        print(f"X{i} = {x1}")

        if(abs(x1-x0)<=tol):
            break

        x0 = x1
        i=i+1
    print(f"\nRoot is : {x1:.4f}")

newrap(x0,tol)
```

Output:

```
C:\Users\Rezwan\PyCharmMiscProject\.venv\Scripts\python
X1 = 0.6200159522472988
X2 = 0.6071206581470717
X3 = 0.6071016481446856

Root is : 0.6071
```

6. Discussion & Conclusion:

The Newton Raphson method successfully finds the root of nonlinear equations with high accuracy and fast convergence. However, the method requires the derivative of the function and may fail if the initial guess is poor or derivative ≈ 0 .

7. Reference:

1. https://en.wikipedia.org/wiki/Newton%27s_method

Experiment: 02

1. Objectives:

- To find the root of a nonlinear equation using the Secant Method.
- To observe how iteration gradually converges to the actual root.

2. Introduction:

The Secant Method is a numerical technique to approximate the root of a nonlinear equation $f(x) = 0$. It is similar to the Newton–Raphson method but does not require the derivative of the function. Instead, it uses two initial approximations and constructs a straight line (secant) between the points $(a, f(a))$ and $(b, f(b))$. The intersection of this secant line with the x-axis is taken as the next approximation. The iterative formula is: $x_{i+1} = \{a * f(b) - b * f(a)\} / \{f(b) - f(a)\}$

This process is repeated until the root is found within the desired tolerance. The method is faster than Bisection and False Position but requires good initial guesses.

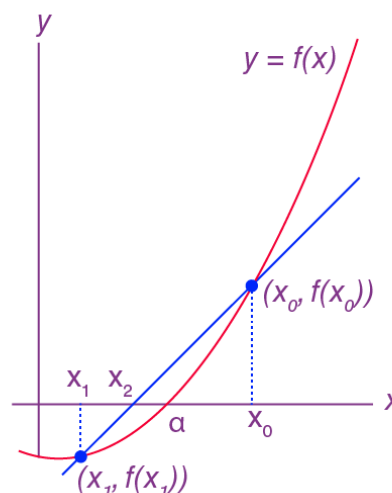


Fig 2: Secant Method.

3. Procedure

Algorithm: Secant Method in Python

1. Start
2. Define function $f(x)$.
3. Input two initial guesses a , b tolerance, and maximum iterations.
4. Repeat until convergence or max iterations:
5. Compute $f(a)$, $f(b)$.
6. Find new approximation:
$$c = \{ a * f(b) - b * f(a) \} / \{ f(b) - f(a) \}$$
7. Check if $|f(c)| \leq \text{tolerance}$ then stop.
8. Else, set $a = b$, $b = c$.
9. End.

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** PyCharm Community Edition 20251.1.1

5. Experiment Implementation:

Code:

```
def f(x):  
    return x ** 3 - 2 * x - 5  
  
a = 2  
b = 3  
tol = 0.00005  
max = 100  
  
def Secant(a, b, tol, max):  
  
    print(f"{'Iteration':<12}{'a':<12}{'b':<12}{'f(a)':<12}  
{'f(b)':<12}{'c':<12}{'f(c)':<12}")  
    print("-" * 82)  
  
    i = 1  
    while i <= max:  
        fa = f(a)  
        fb = f(b)  
        c = (a * fb - b * fa) / (fb - fa)  
        fc = f(c)  
  
    print(f"{'i':<12}{'a':<12.6f}{'b':<12.6f}{'fa':<12.6f}{'fb':<12.  
6f}{'c':<12.6f}{'fc':<12.6f}")
```

```

        if abs(fc) <= tol:
            break

        a = b
        b = c
        i += 1

    return (a * fb - b * fa) / (fb - fa)

root = Secant(a, b, tol, max)
if root is not None:
    print(f"\nRoot is : {root:.6f}")

```

Output: C:\Users\Rezwan\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\Rezwan\PyCharm

Iteration	a	b	f(a)	f(b)	c	f(c)
1	2.000000	3.000000	-1.000000	16.000000	2.058824	-0.390800
2	3.000000	2.058824	16.000000	-0.390800	2.081264	-0.147204
3	2.058824	2.081264	-0.390800	-0.147204	2.094824	0.003044
4	2.081264	2.094824	-0.147204	0.003044	2.094549	-0.000023

Root is : 2.094549

Process finished with exit code 0

6. Discussion & Conclusion:

The Secant Method successfully found the root near 2.0450 within the given tolerance. It converged faster than the Newton–Raphson Method in terms of simplicity, since it does not require the derivative of the function, but its accuracy depends strongly on good initial guesses.

7. Reference:

1. <https://byjus.com/maths/secant-method/>